



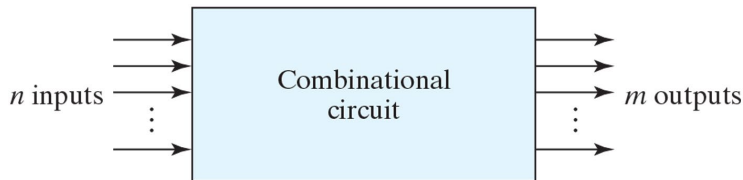
디지털공학

5 주차

4 조합 논리

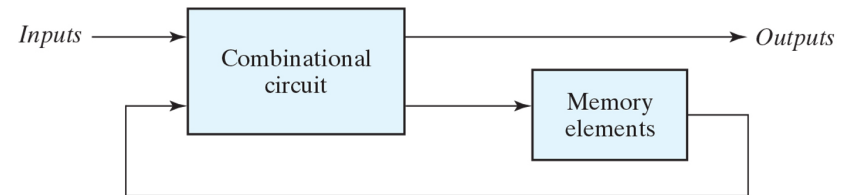
- 논리회로는 조합회로(Combinational logic) 또는 순차회로(Sequential Logic)로 구분

❖ 조합논리

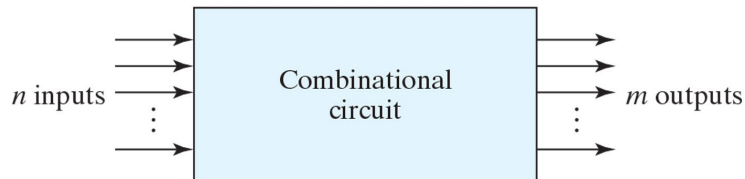


- 부울 함수들에 의해 정의된 논리 연산 수행
- 논리 게이트의 연결로 이루어짐
- 입력단의 신호에 의해 출력을 생성
- 4장에서 다룸

❖ 순차논리



- 논리 게이트 + 저장회로
- 출력 값은 현재 입력 및 저장회로에 저장된 이전 값의 조합으로 결정
- 즉, 시간 순서에 따른 입력 값과 저장된 내부 상태 값에 따라 달라짐
- 5장/8장에서 상세히 다룰 예정



- 규정된 입력 변수의 조합으로 규정된 출력이 나오도록 논리 게이트를 연결한 논리회로를 의미함
- 저장장치 미 포함
- 출력 레벨은 항상 입력 레벨의 조합으로만 결정됨
- 본 장에서는 조합논리회로의 분석, 설계와 관련된 사항을 다룸
- 조합논리회로 **분석, 설계** 기법을 통해 가산기, 비교기, 인코더, 디코더 등 다양한 조합 논리회로의 동작을 이해

❖ 조합논리 회로 분석

- 이미 구성되어 있는 조합논리 회로의 동작에 대한 분석을 통해 진리표 도출

Step-1) 입력 변수와 연결된 게이트(입력과 가까운 게이트 레벨) 출력에 기호를 할당하고 부울 함수 도출

Step-2) 다음 게이트 레벨에 대한 부울 함수 도출

Step-3) 최종 출력에 대한 부울 함수가 도출될 때 까지 Step-2 반복

Step-4) 각 게이트 레벨 마다 도출된 부울 함수를 대입하여 부울 함수를 정리하고 진리표 작성

[예제] 다음 논리회로도 분석

Step-1) 입력 변수와 연결된 게이트(입력과 가까운 게이트 레벨) 출력에 기호를 할당하고 부울 함수 도출

$$\begin{aligned} F_2 &= AB+AC+BC \\ T_1 &= A+B+C \\ T_2 &= ABC \end{aligned}$$

Step-2) 다음 게이트 레벨에 대한 부울 함수 도출

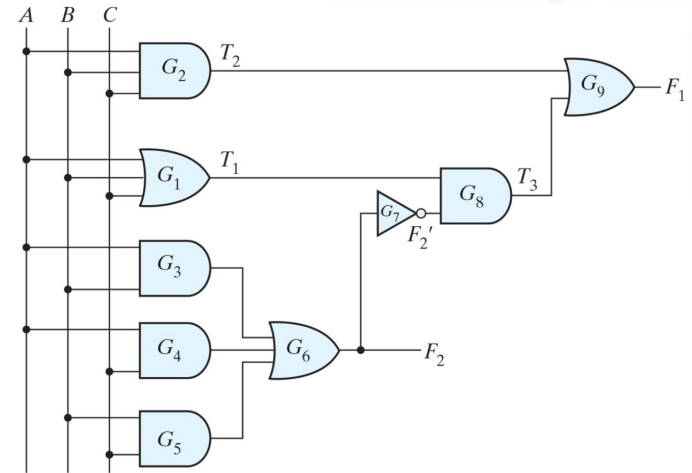
$$\begin{aligned} T_3 &= F_2' T_1 \\ F_1 &= T_2 + T_3 \end{aligned}$$

Step-3) 최종 출력에 대한 부울 함수가 도출될 때까지 Step-2 반복

$$F_1 = T_2 + T_3$$

Step-4) 각 게이트 레벨마다 도출된 부울 함수를 대입하여 부울 함수를 정리하고 진리표 작성

$$\begin{aligned} F_1 &= T_2 + T_3 = F_2' T_1 + ABC = (AB+AC+BC)'(A+B+C) + ABC \\ &= A'BC' + A'B'C + AB'C' + ABC \end{aligned}$$



A	B	C	F ₂	F ₂ '	T ₁	T ₂	T ₃	F ₁
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

❖ 조합논리 회로 설계

- 설계 목적을 규정한 설계 사양을 기반으로 논리 회로를 작성하는 것을 목적으로 함

Step-1) 설계 사양의 입출력을 분석하고 각각의 입출력에 기호 할당

Step-2) 입력과 출력 사이의 관계를 정의하는 진리표 작성

Step-3) 입력 변수의 함수로 각 출력에 대한 간략화 된 부울 함수 정의

Step-4) 논리회로도 작성 및 검증

[예제] 다음 진리표에 대한 논리회로 설계

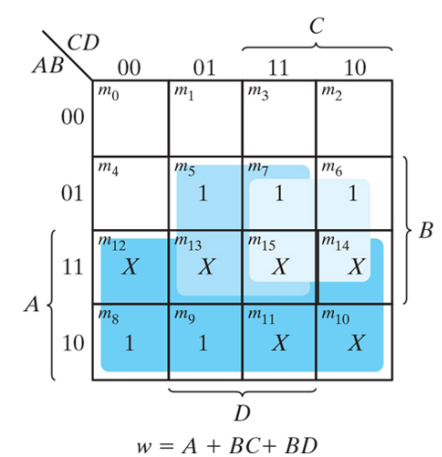
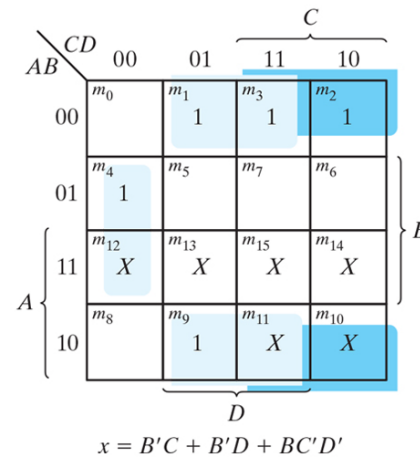
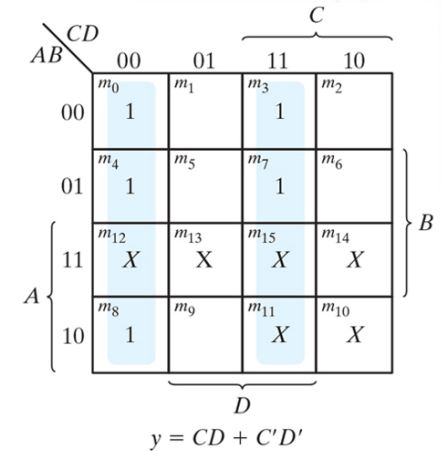
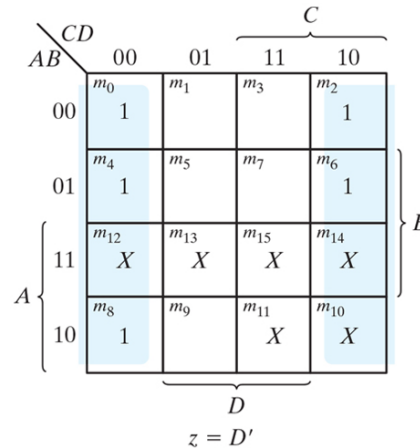
Input BCD				Output Excess-3 Code			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Step-1) 설계 사양의 입출력을 분석하고 각각의 입출력에 기호 할당

Step-2) 입력과 출력 사이의 관계를 정의하는 진리표 작성

Step-3) 입력 변수의 함수로 각 출력에 대한 간략화된 부울 함수 정의

Step-4) 논리회로도 작성 및 검증

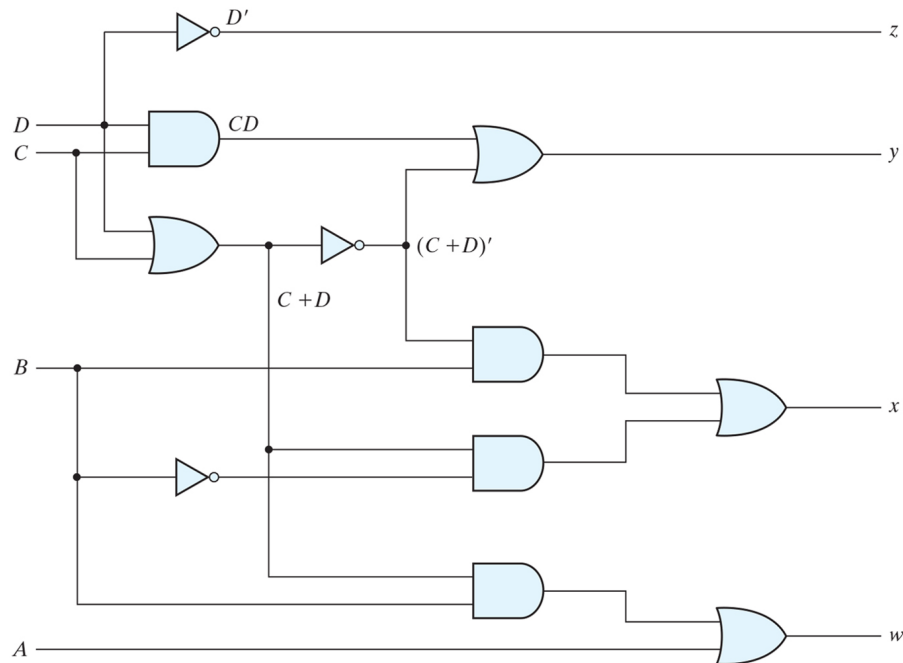


$$Z = D'$$

$$Y = CD + C'D' = CD + (C+D)'$$

$$X = B'C + B'D + BC'D' = B'(C+D) + BC'D' = B'(C+D) + B(C+D)'$$

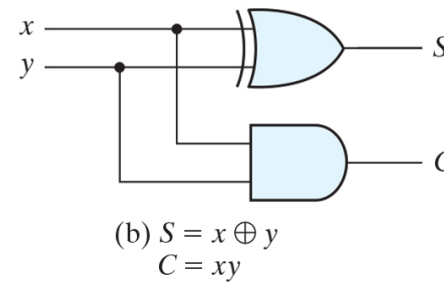
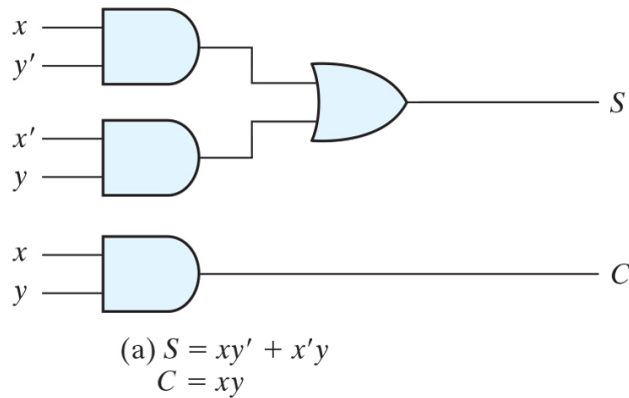
$$W = A + BC + BD = A + B(C+D)$$



❖ 가산기 – (반 가산기)

- 2개의 입력과 2개의 2진 출력으로 정의
- 출력은 합(sum), 캐리(Carry)

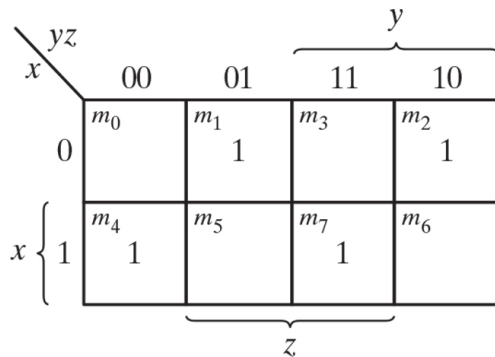
x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



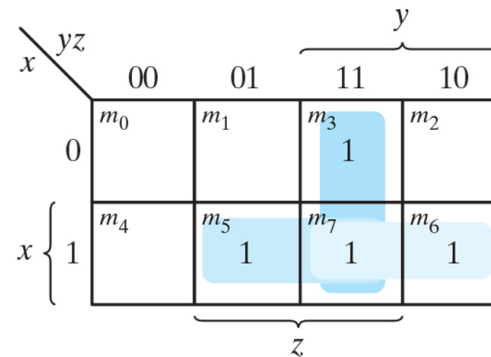
❖ 가산기 – (전 가산기)

- 3개의 2진 입력과 2개의 2진 출력으로 정의
- 2-bit 이상의 덧셈 연산에서는 각 자리에서 발생하는 캐리(올림)에 대한 처리가 필요
- 출력은 합(sum), 캐리(Carry)
- 반가산기 2개로 전가산기 구현 가능

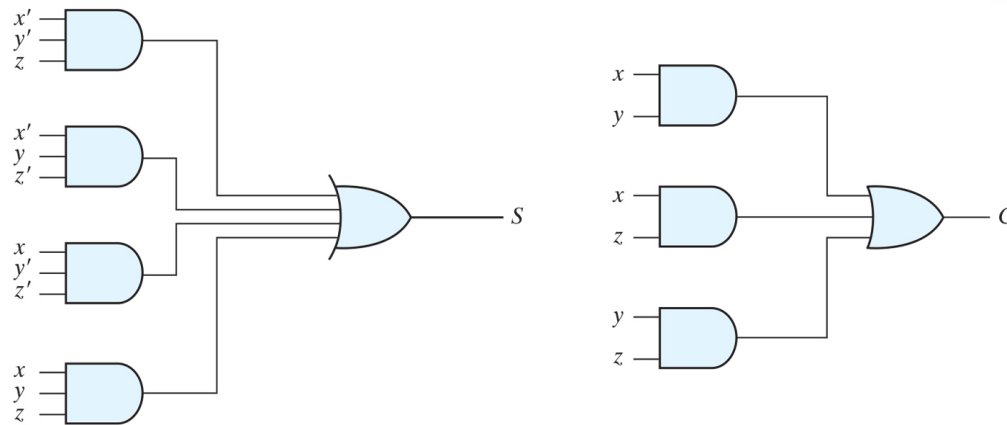
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$(a) S = x'y'z + x'yz' + xy'z' + xyz$$

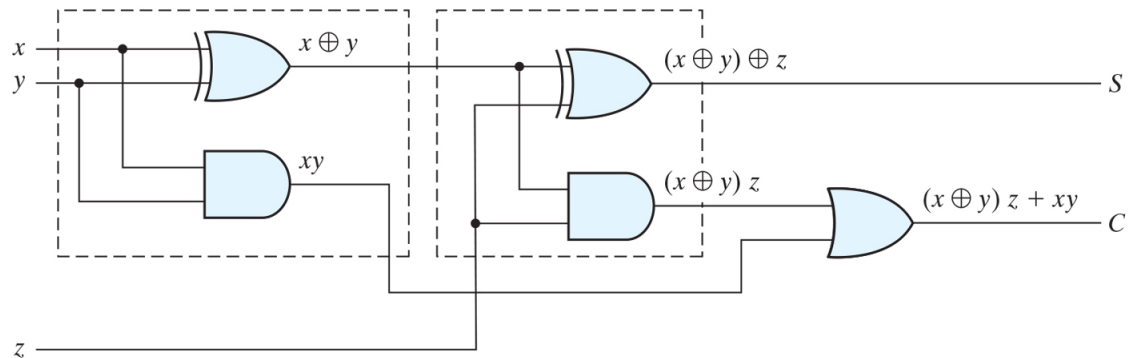


$$(b) C = xy + xz + yz$$

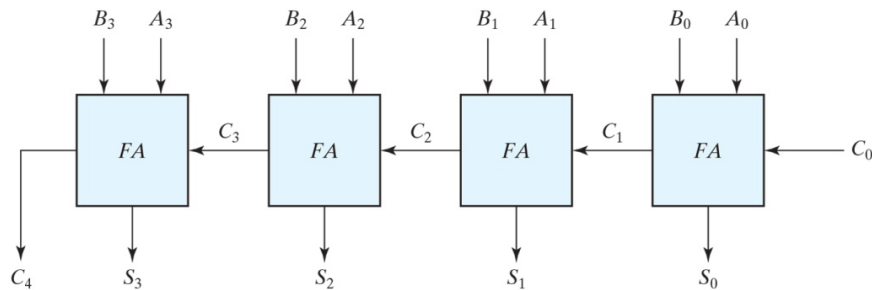


$$\begin{aligned}
 S &= xy'z' + x'yz' + xyz + x'y'z \\
 &= z'(xy' + x'y) + z(xy + x'y') = z'(xy' + x'y) + z(xy' + x'y)' \\
 &= z'(x \oplus y) + z(x \oplus y)' \\
 &= z \oplus x \oplus y
 \end{aligned}$$

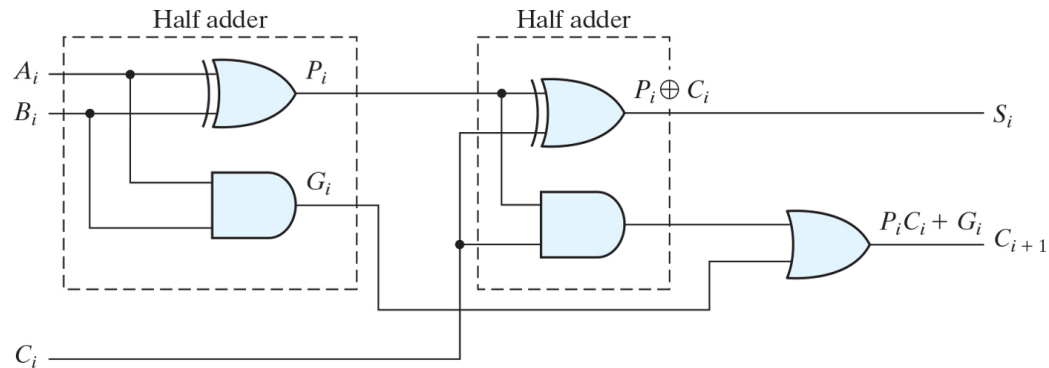
$$\begin{aligned}
 C &= xy + yz + xz = xyz + xyz' + xyz + x'y'z + xyz + xy'z = xyz + xyz' + x'y'z + xy'z \\
 &= xy(z + z') + z(x'y + xy') = xy + z(x \oplus y)
 \end{aligned}$$



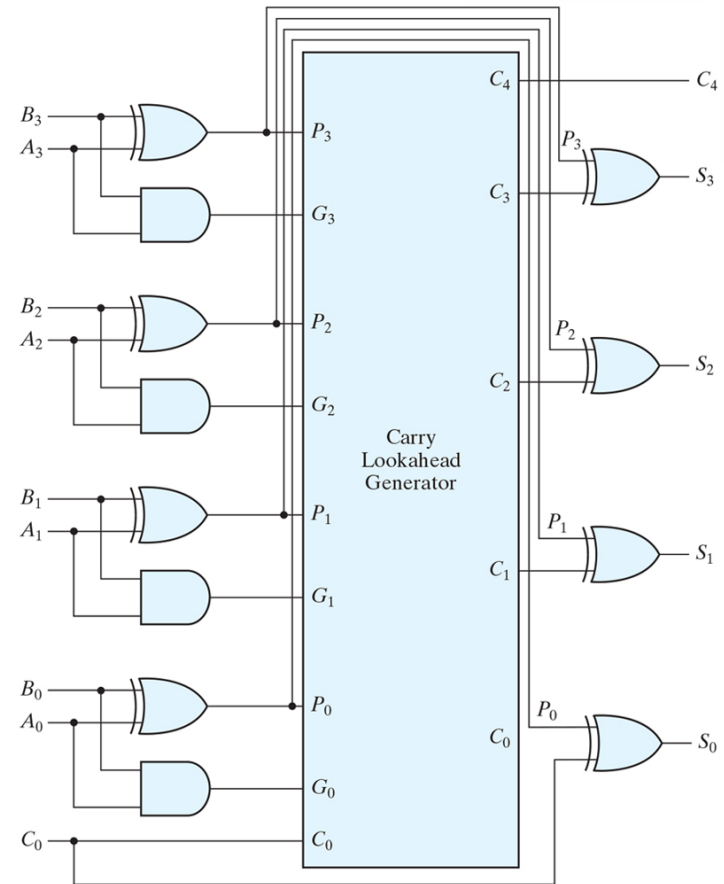
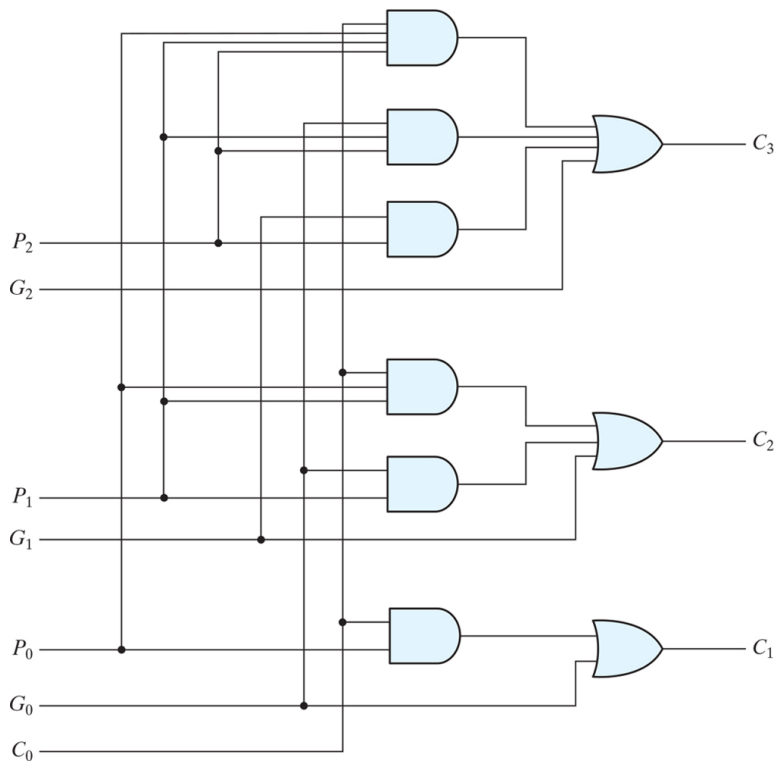
❖ 가산기 - (2진 가산기)



Subscript i :	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}



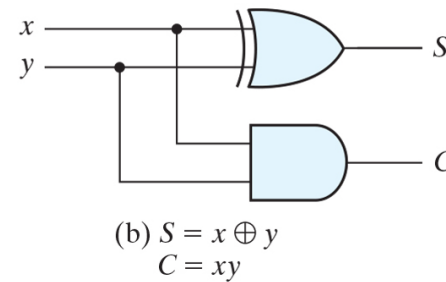
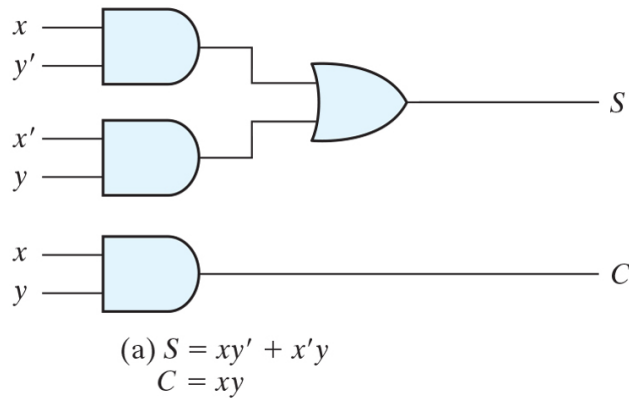
❖ 캐리 룩어헤드 가산기



❖ 가산기 – (반 가산기)

- 2개의 입력과 2개의 2진 출력으로 정의
- 출력은 합(sum), 캐리(Carry)

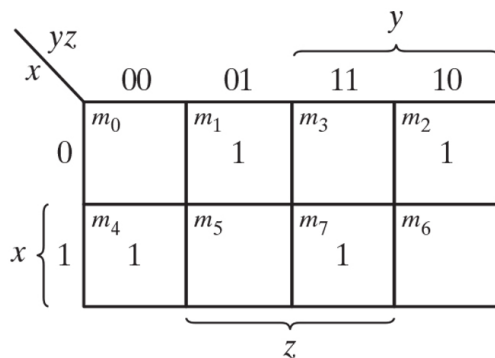
x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



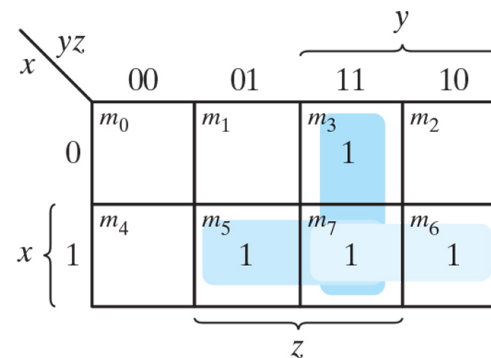
❖ 가산기 – (전 가산기)

- 3개의 2진 입력과 2개의 2진 출력으로 정의
- 2-bit 이상의 덧셈 연산에서는 각 자리에서 발생하는 캐리(올림)에 대한 처리가 필요
- 출력은 합(sum), 캐리(Carry)
- 반가산기 2개로 전가산기 구현 가능

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

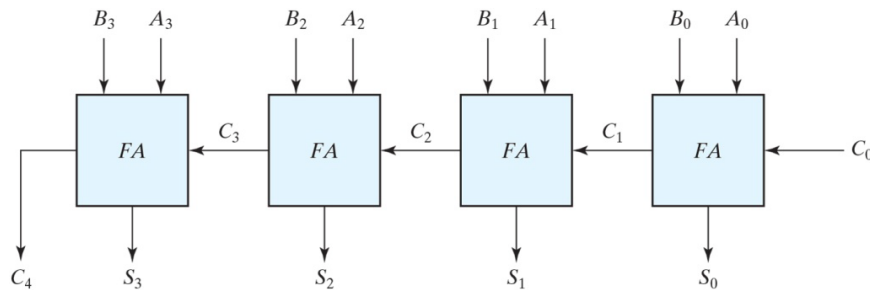


$$(a) S = x'y'z + x'yz' + xy'z' + xyz$$

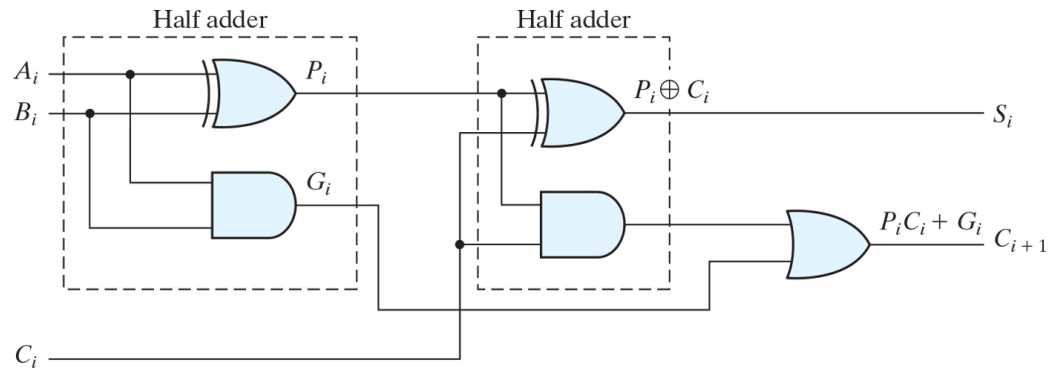


$$(b) C = xy + xz + yz$$

❖ 가산기 - (2진 가산기)

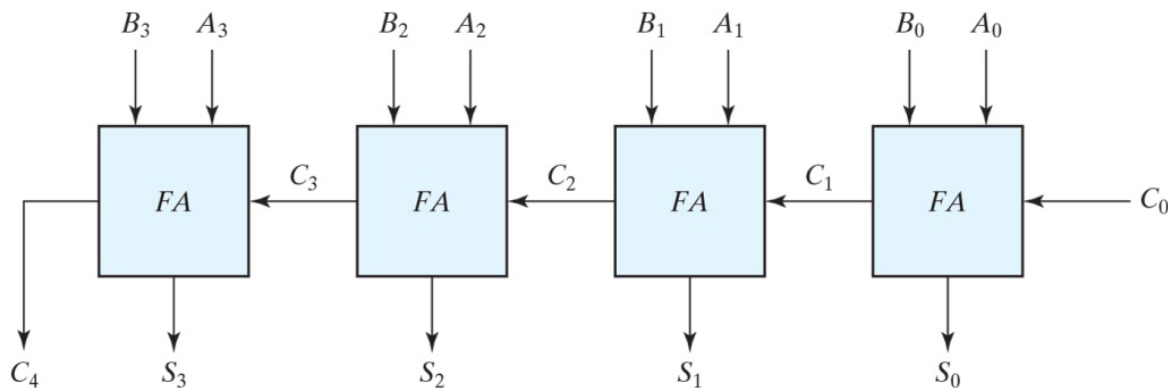


Subscript i :	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}



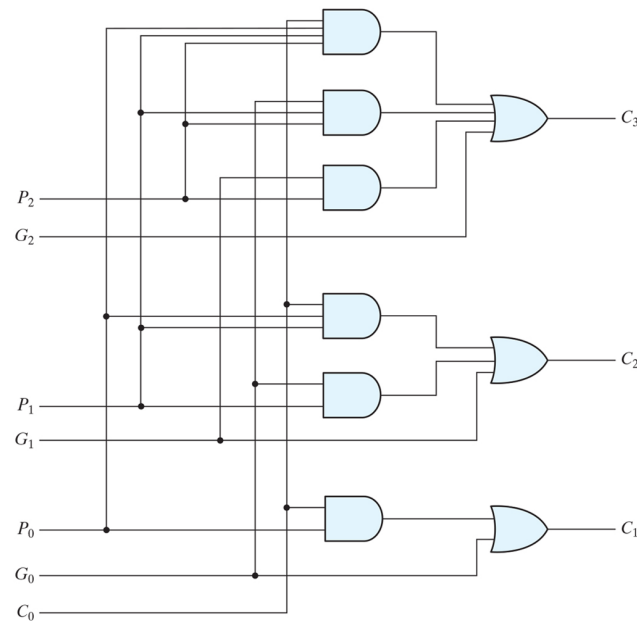
❖ 리플 가산기

- 여러 개의 전가산기로 임의의 비트 수를 더하는 가산기를 만들 때, 하위 비트의 Carry를 상위 비트 연산으로 전달하는 방식을 칭함
- 설계가 간단하다는 장점이 있으나, 하위 비트의 연산이 차례로 연산이 되어야 해 Ripple(물결) 가산기로 불리기도 함
- 상기와 같은 특성으로 비트 수가 많아 질 수록 연산이 느려지는 단점이 있으며, 연산 속도 개선을 위해 Carry Look-ahead 가산기 등을 사용하기도 함



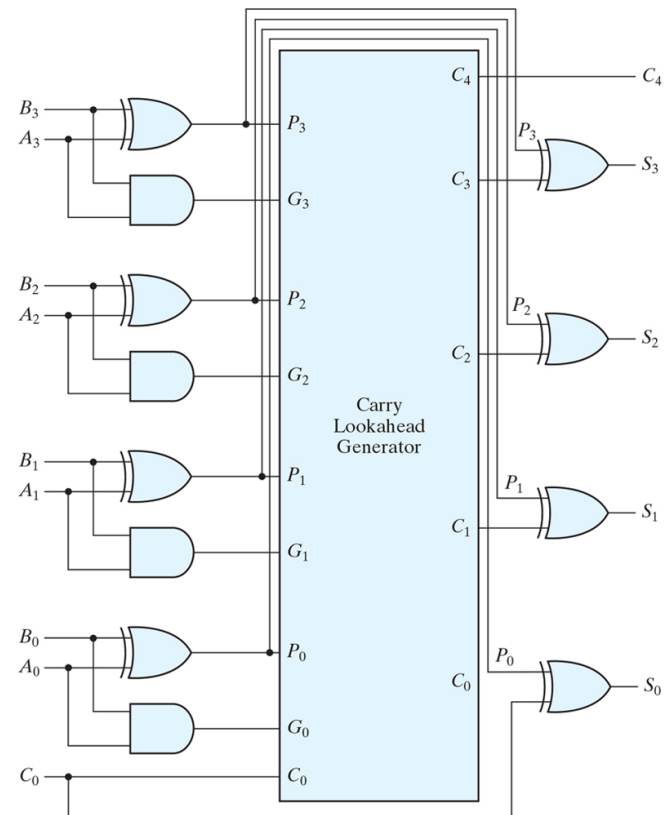
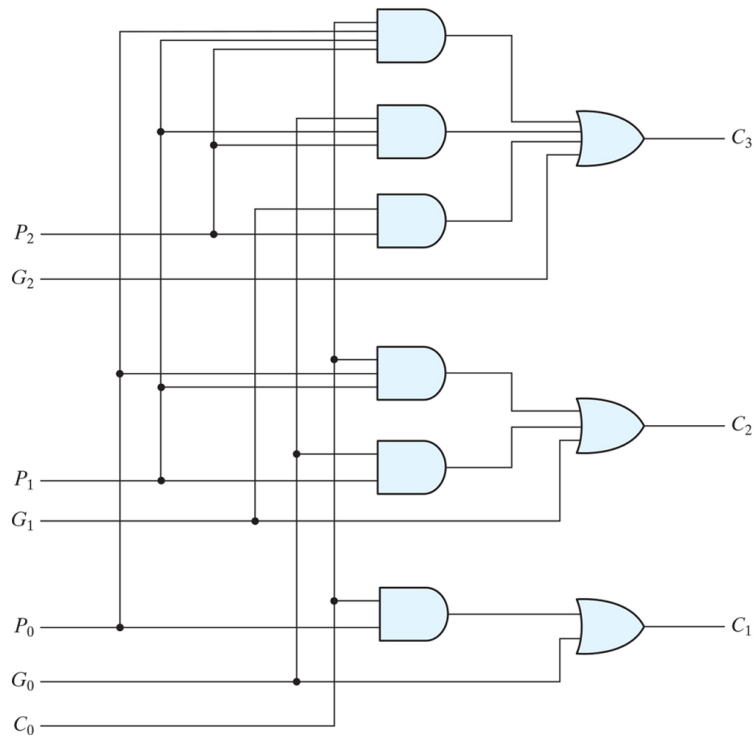
❖ Carry Look-ahead 가산기

- 덧셈의 고속처리를 위해 Carry(자리올림) 신호를 따로 계산하는 회로를 가지는 가산기를 설계
- 자리올림수의 생성(Generate)와 전달(Propagate)의 개념으로 정의
- 자리올림수가 발생하는 경우는 해당 자리의 합에 의해 발생하는 경우, 해당 자리의 합과 하위 자리의 올림 수와의 합에 의해 발생하는 경우가 존재



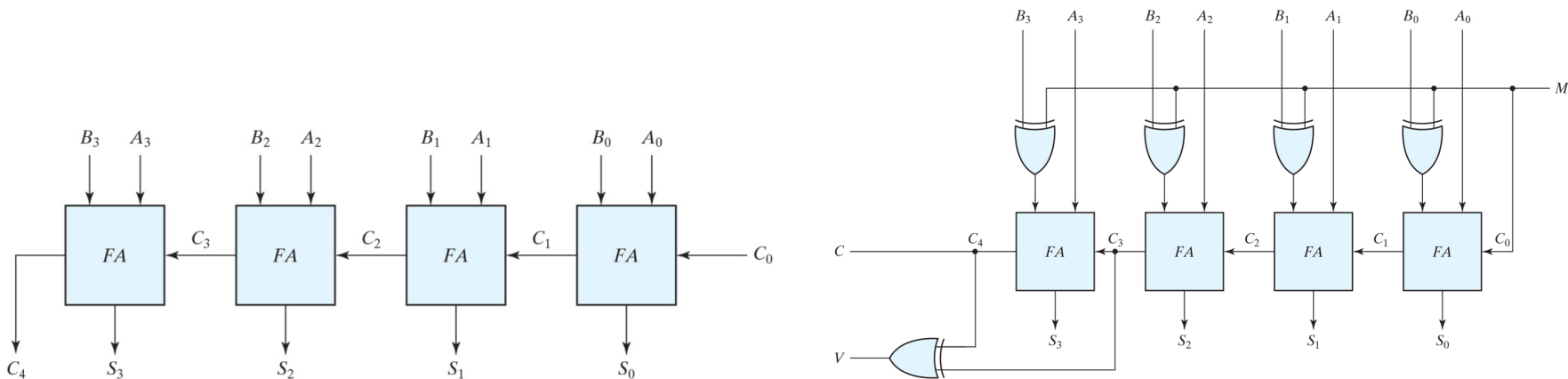
[Carry Look-ahead 회로]

- 각 자리의 합(S)과 자리 올림수(C)를 동시에 연산하고 해당 결과에 대한 합을 구하여 가산기를 구현하여 덧셈 속도 향상
- 속도는 개선 되지만, 회로의 복잡도가 증가하고, 이로 인해 회로 구성에 대한 비용 및 전력 소모량 증가



❖ 감산기

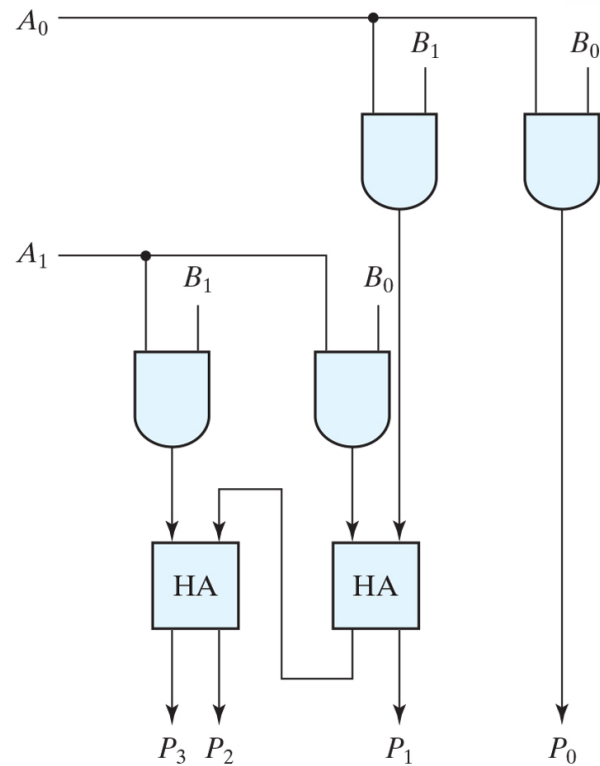
- 논리회로 시스템에서 뺄셈은 2의 보수를 취해서 덧셈 처리
- 2의 보수는 1의 보수를 취한 후 1을 더해서 생성
- XOR 및 입력신호 M을 이용해 2의 보수 구현 (M=0일 경우 가산기, M=1일 경우 감산기)



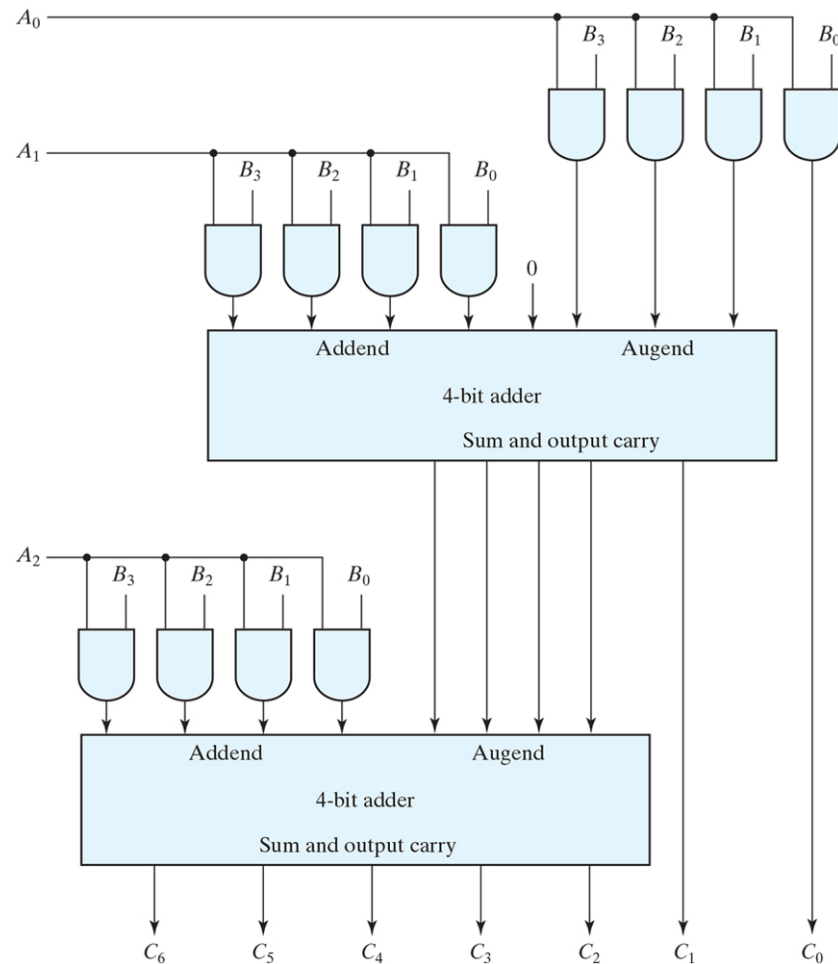
❖ 곱셈기

- 2-bit A와 B의 곱을 구한다고 가정할 경우
- 출력은 4-bit (2+2)

		B_1	B_0
	A_1	$A_1 B_1$	$A_1 B_0$
	A_0	$A_0 B_1$	$A_0 B_0$
P_3	P_2	P_1	P_0



- 3-bit A와 4-bit B의 곱을 구한다고 가정할 경우
- 출력은 7-bit (3+4)



❖ 비교기

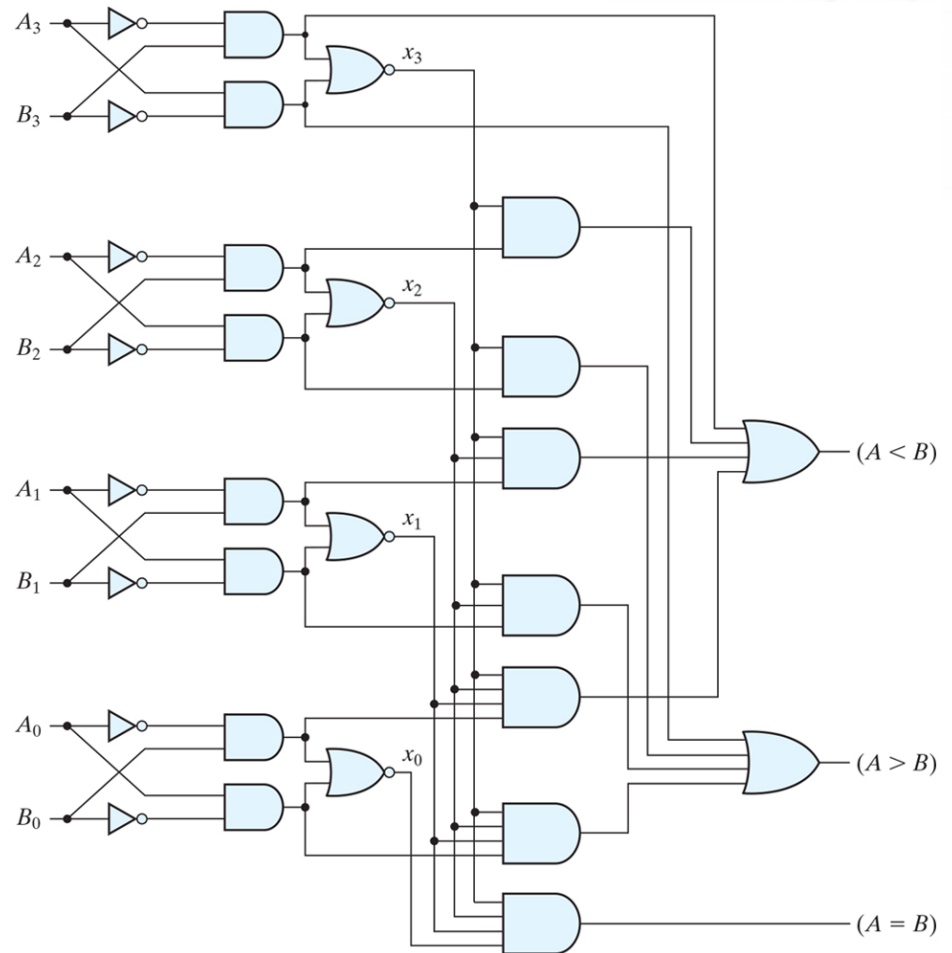
- $A = A_3 A_2 A_1 A_0$ $B = B_3 B_2 B_1 B_0$
- $x_i = A_i B_i + A'_i B'_i$ (i번째 비트가 같을 경우 1)
- 모든 비트가 같으면 $x_3 x_2 x_1 x_0 = 1$ ($A=B$)

- ($A > B$)

$$A_3 B'_3 + A_2 B'_2 x_3 + A_1 B'_1 x_3 x_2 + A_0 B'_0 x_3 x_2 x_1$$

- ($A < B$)

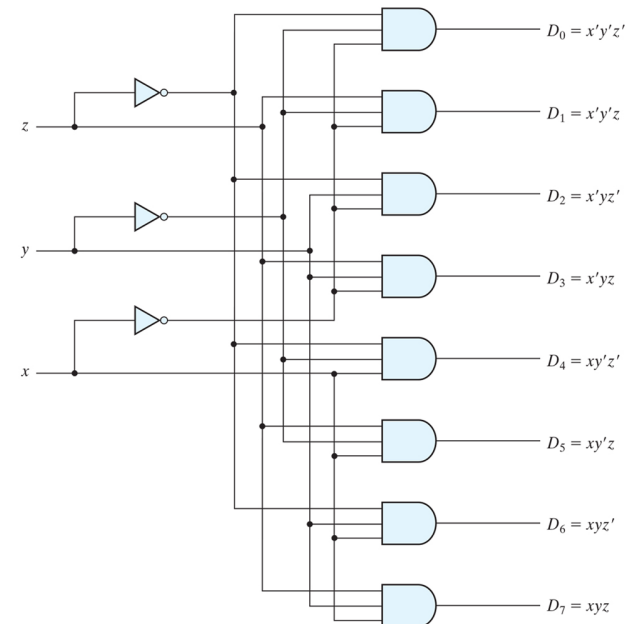
$$A'_3 B_3 + A'_2 B_2 x_3 + A'_1 B_1 x_3 x_2 + A'_0 B_0 x_3 x_2 x_1$$



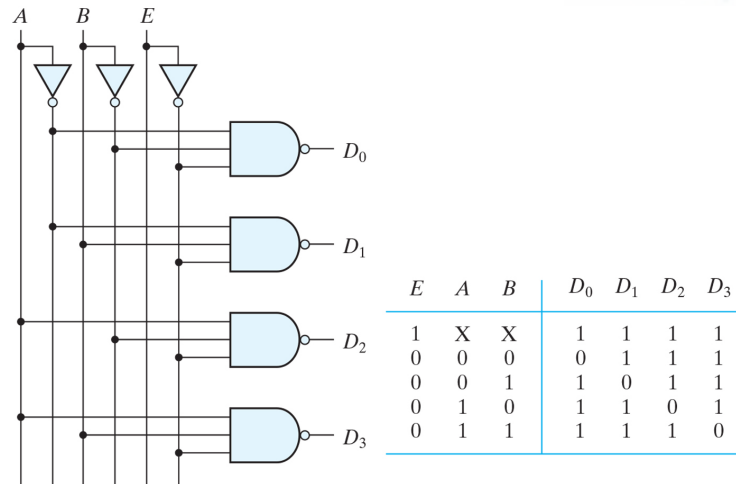
❖ 디코더

- n-bit의 2진 입력을 최대 2^n 개의 출력으로 변환하는 조합논리 회로
- 입력 중 사용되지 않는 조합이 있을 경우 출력의 수는 2^n 개 보다 작을 수 있음

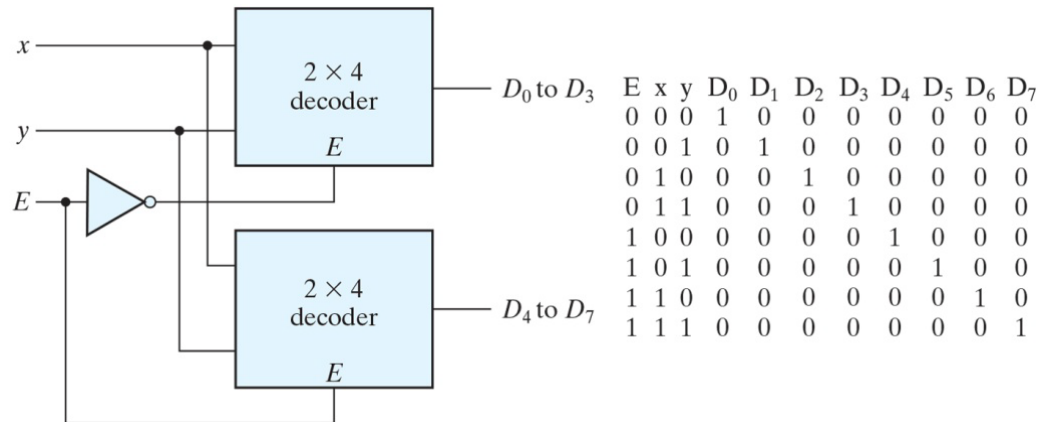
Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1



- Enable 입력 신호가 있는 2x4 디코더



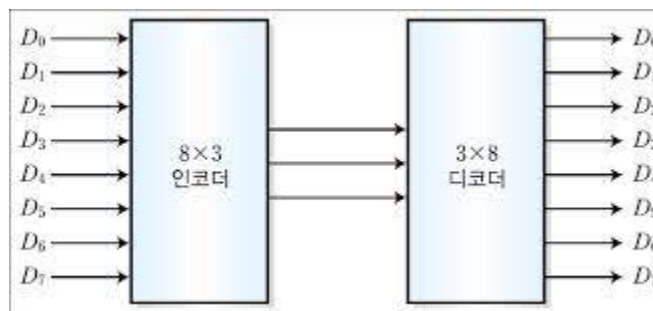
- 2x4 디코더 2개를 이용해 한 개의 3x8 디코더 설계



❖ 인코더

- 2^n 개의 입력으로 n 개의 출력으로 변환하는 조합논리 회로

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

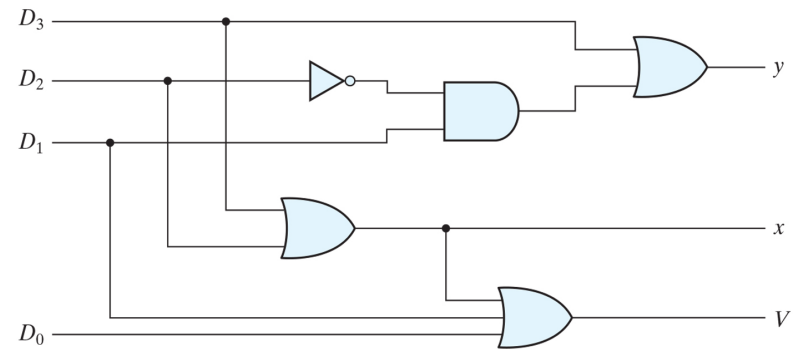


- 우선순위 인코더

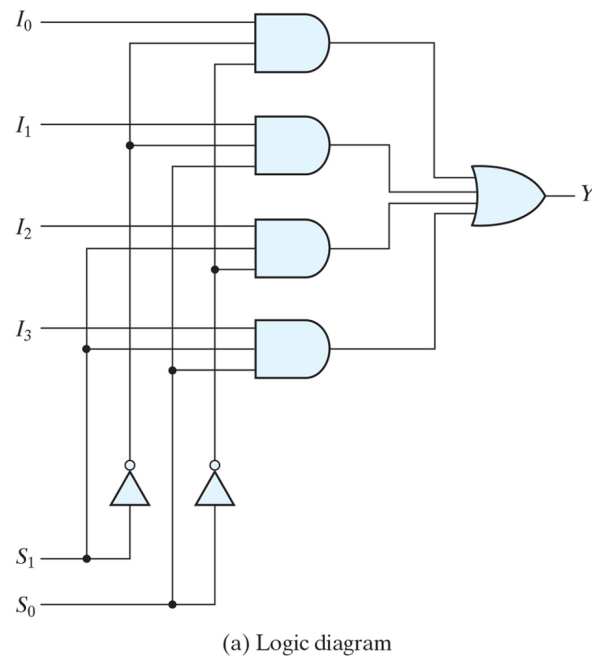
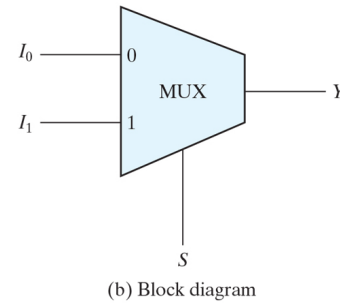
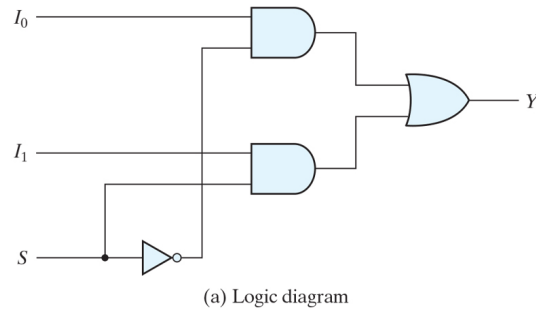
Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

D_0D_1		D_2D_3				D_1
		00	01	11	10	
D_0	00	m_0 X	m_1 1	m_3 1	m_2 1	}
	01	m_4 1	m_5 1	m_7 1	m_6 1	
	11	m_{12} 1	m_{13} 1	m_{15} 1	m_{14} 1	
	10	m_8 1	m_9 1	m_{11} 1	m_{10} X	
		D_3				
		$x = D_2 + D_3$				

D_0D_1		D_2D_3				D_1
		00	01	11	10	
D_0	00	m_0 X	m_1 1	m_3 1	m_2 1	}
	01	m_4 1	m_5 1	m_7 1	m_6 1	
	11	m_{12} 1	m_{13} 1	m_{15} 1	m_{14} 1	
	10	m_8 1	m_9 1	m_{11} 1	m_{10} 1	
		D_3				
		$y = D_3 + D_1D'_2$				



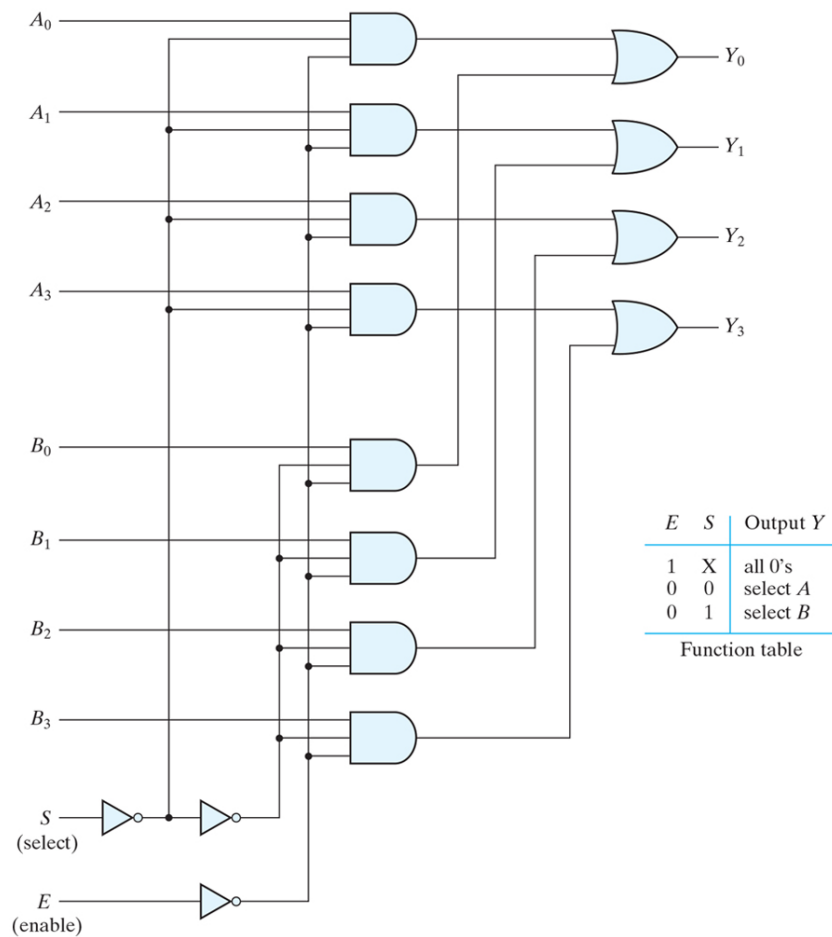
❖ 멀티플렉서



S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table



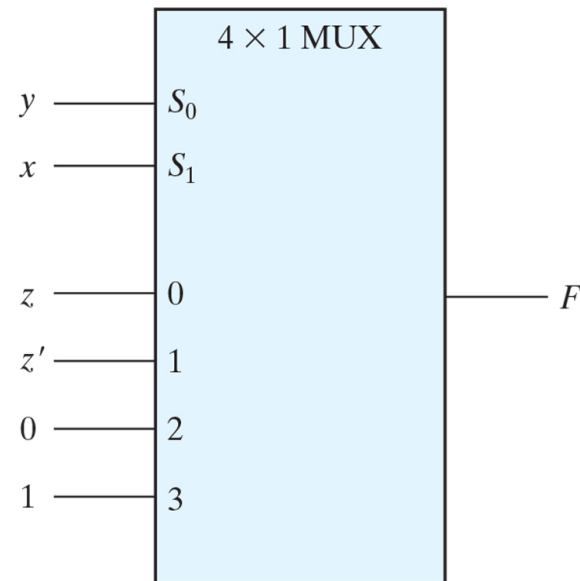


- 멀티플렉서를 사용한 부울함수 구현

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) Truth table

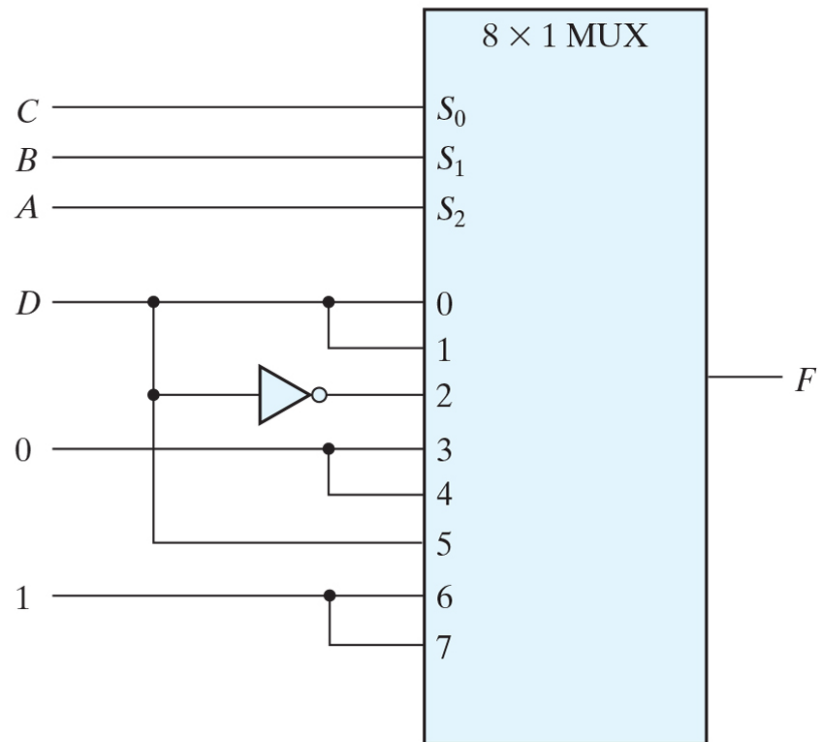


(b) Multiplexer implementation



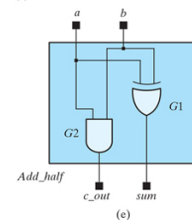
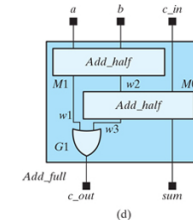
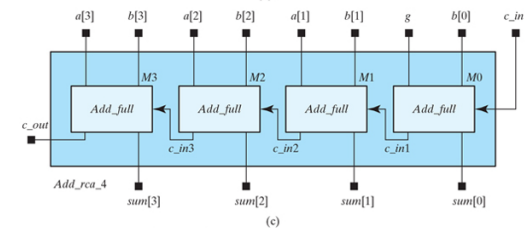
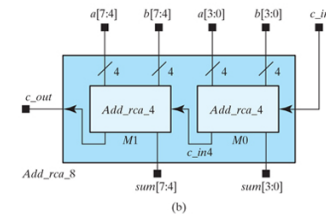
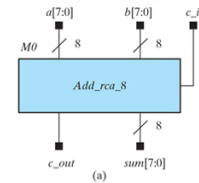
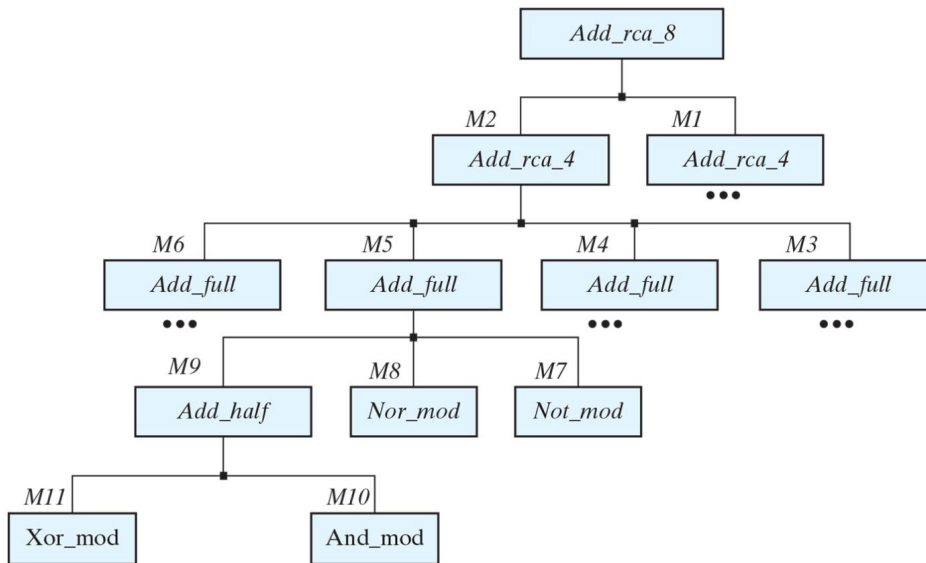
$$F(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$$

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	

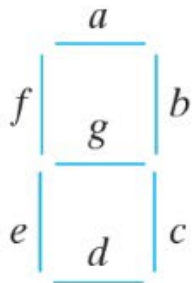


❖ 계층적 모델링

- 복잡한 논리회로 설계 시 Top-Down 방식으로 설계
- 예를 들어 8-bit 가산기는 2개의 4-bit 가산기로 설계
- 4-bit 가산기는 4개의 전가산기를 이용하여 설계



- ❖ [과제] 연습문제 4.9) BCD 입력을 7-Segment로 변환하는 디코더를 진리표와 카노맵을 이용하여 설계. 사용하지 않는 6개의 입력 조합은 아무런 표시도 되지 않아야 함.



(a) Segment designation



(b) Numerical designation for display